

ФУНДАМЕНТ IT · ДЕНЬ 16 ИЗ 30

Модули, пакеты и зависимости

Собираем программу из файлов и подключаем готовые библиотеки.

Главная идея

Модуль - обычный Python-файл, который можно импортировать. Пакет объединяет связанные модули в папке. Это помогает делить большой проект по ответстве...



```
from pathlib import Path

project_file = Path("data") /
"users.json"
print(project_file)
```

План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить
термины

3

Разобрать код

4

Попробовать
самому

5

Собрать мини-
задачу

Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

Смысл темы простыми словами

Сначала бытовая идея, потом код.

Идея 1

Модуль - обычный Python-файл, который можно импортировать. Пакет объединяет связанные модули в папке. Это помогает делить большой проект по ответственности.

Идея 2

Зависимость - сторонний пакет, необходимый проекту. pip устанавливает пакет, а файл requirements.txt фиксирует список и часто версии зависимостей.

Идея 3

Версии важны: новая версия может добавить возможности или изменить совместимость. Устанавливать библиотеки нужно из доверенных источников и внутри виртуального окружения.

Термины дня

Эти слова нужно узнавать в коде и документации.

module

Python-файл, код которого можно импортировать

package

набор связанных Python-модулей

import

подключение доступного модуля в программу

dependency

внешняя библиотека, от которой зависит проект

requirements

описание зависимостей проекта и их версий

version

идентификатор конкретного выпуска программы или пакета

Код дня

Минимальный пример без лишней сложности.

```
day16_example.py

from pathlib import Path

project_file = Path("data") / "users.json"
print(project_file)
```

Что ищем

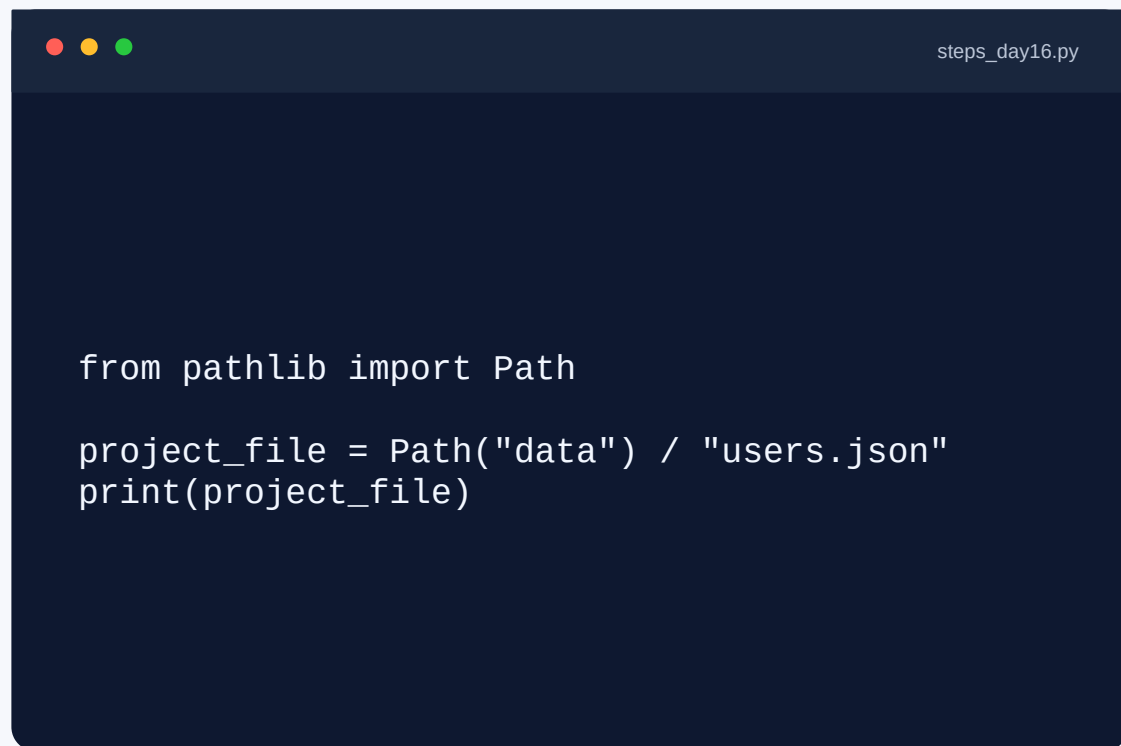
Где данные? Где действие? Где результат?

Главный вопрос

Можешь объяснить каждую строку обычным языком?

Разбор кода по шагам

Не переписываем вслепую.



```
from pathlib import Path

project_file = Path("data") / "users.json"
print(project_file)
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.

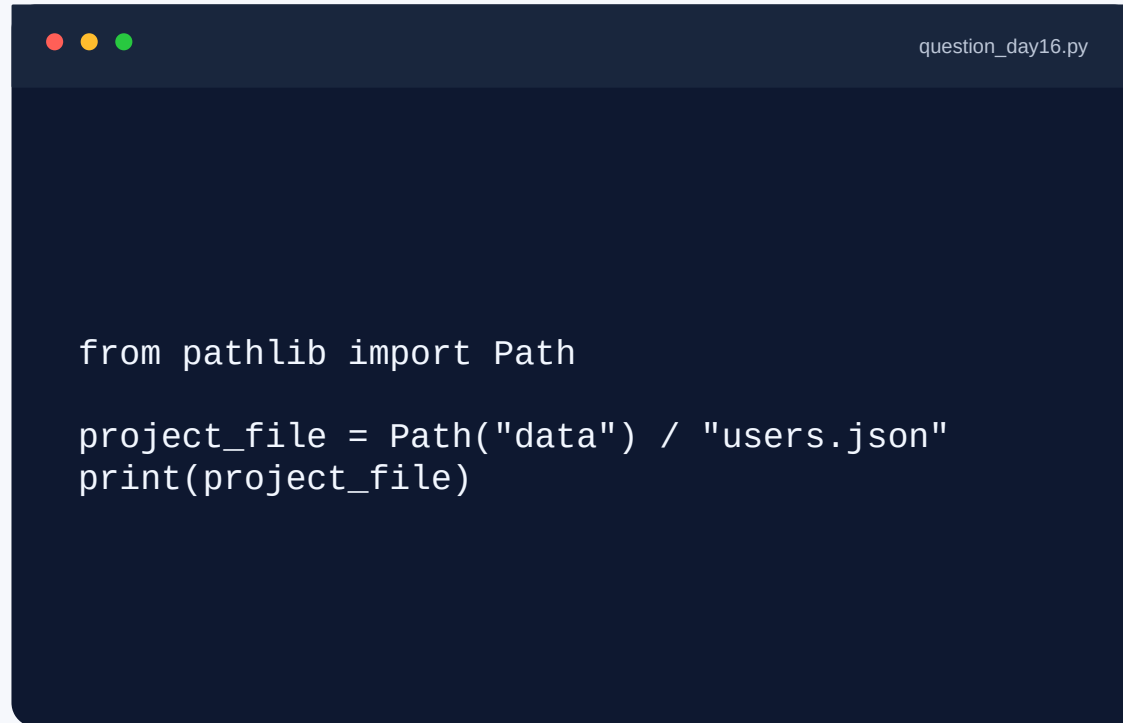


Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

Практика: что произойдет?

Ответь без запуска кода.



```
from pathlib import Path

project_file = Path("data") / "users.json"
print(project_file)
```

Вопрос

Что программа сделает после запуска?

Подсказка

Ищи последовательность: входные данные -> действие -> результат.

Ответ и объяснение

Сравни со своим рассуждением.

Ожидаемая логика

Этот пример относится к теме: Модули, пакеты и зависимости. Собираем программу из файлов и подключаем готовые библиотеки.

Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

Практика: сделай руками

Небольшое задание по теме дня.

Задание

Создай `helpers.py` и импортируй функцию в `main.py`.

Шаг 1

Создай простой файл или черновик.

Шаг 2

Напиши минимальный рабочий вариант.

Шаг 3

Запусти и проверь результат.

Мини-тест

Выбери ответ и объясни почему.

Вопрос

Для чего нужен requirements.txt?

A

Для хранения пользовательских паролей

B

Для описания зависимостей проекта

C

Для запуска операционной системы

D

Для замены исходного кода

Ответ мини-теста

Проверяем не память, а понимание.

Правильный ответ

В: Для описания зависимостей проекта

Объяснение

requirements.txt помогает воспроизвести набор Python-зависимостей проекта.

Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

Как исправить

После каждого термина придумай свой пример и найди его в коде.

Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

Условие

Создай `helpers.py` и импортируй функцию в `main.py`.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

Разбор решения

После попытки сравни с этим подходом.

```
from pathlib import Path

project_file = Path("data") / "users.json"
print(project_file)
```

Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

Проверка

Запусти пример и объясни результат вслух.

Задание со звездочкой

Немного сложнее, но без новой темы.

Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

Итог дня

Что должно остаться после урока.

- объяснять тему "Модули, пакеты и зависимости" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

Домашнее задание

Создайте модуль `helpers.py` с одной функцией, импортируйте её в `main.py`. Посмотрите список пакетов командой `pip list`.